

SDP Project — Setup Notes (Repo & Git Workflow)

These notes cover everything done so far to set up the project repository in a "production style" — folder structure, Git workflow, branch protection, and the issues faced along the way. Useful for personal reference and for explaining the process in an interview.

1. What Was Built

A repository scaffold (before writing any actual project code) with a production-style structure:

```
sdp-project/  
├── services/           # backend microservices (ingestion, processing, api)  
├── frontend/          # dashboard/UI  
├── infra/  
│   ├── terraform/     # Infrastructure as Code  
│   └── k8s/            # Kubernetes manifests  
├── monitoring/  
│   ├── prometheus/  
│   └── grafana/  
├── .github/workflows/  # CI/CD pipeline (GitHub Actions)  
├── docs/              # architecture, setup guide, API spec, contributing guide  
├── scripts/  
├── tests/  
├── docker-compose.yml  # local dev environment  
└── README.md
```

Why this matters (interview point): Instead of dumping all code in one folder, the repo is organized the way real companies structure projects — separating application code, infrastructure code, monitoring, and documentation. This makes the project look and behave like a real production system, not just a college assignment.

2. Git & GitHub Concepts Used

a) Git Branching Strategy

Instead of writing code directly on the `main` branch, the workflow is:

```
main    → always stable, deployable code (protected)
└─ dev   → integration branch where features come together
    └─ feature/<name> → one branch per task/feature
```

Why: If `main` always has a broken/half-done feature, the project can look broken during evaluation/demo. Feature branches let you work on something without risking the stable version.

b) Pull Requests (PR) — Core Concept

A **Pull Request** is a request to merge changes from one branch into another (e.g., `feature/xyz` → `main`), instead of pushing directly.

Why PRs matter, even solo:

- Acts as a **checkpoint** before code reaches the stable branch.
- Automatically triggers **CI checks** (tests, build) — code isn't merged unless it passes automated checks.
- Creates a **clear history** of what changed and why (each PR has a title/description).
- Mirrors exactly how real engineering teams work — this is one of the most common things interviewers check understanding of.

Interview line: *"Even working solo, I used a PR-based workflow instead of pushing directly to main, because it keeps the main branch stable, runs automated checks before merge, and creates a clean audit trail of changes — similar to how production teams work."*

c) Branch Protection Rules

Set up on GitHub (Settings → Branches) to protect `main`:

- **Require a pull request before merging** → blocks direct pushes to `main`.
- **Require status checks to pass before merging** → CI (GitHub Actions) must pass first.
- Initially also enabled "Require approvals" — this caused a problem (see below).

d) CI/CD (GitHub Actions)

A basic CI pipeline (`.github/workflows/ci.yml`) was set up to:

- Install dependencies
- Run tests/lint
- Build Docker images for each service

Runs automatically on every push/PR — so broken code gets flagged before merge.

3. Problems Faced & How They Were Solved

Problem 1: Nested folder issue

What happened: When the scaffolded project was zipped and pushed to GitHub, the repo ended up with a structure like `repo-root/sdp-project/...` instead of `repo-root/...` — an unnecessary extra folder layer. This meant the README wasn't visible on the GitHub homepage, and paths were unnecessarily nested.

Root cause: The zip extracted into a folder with the same name as its parent, creating duplication.

Fix: Used `git mv` to move every file/folder from inside the nested folder up to the actual repository root, then deleted the now-empty nested folder, committed, and pushed.

Interview line: *"I had a folder-nesting issue after extracting a zip archive into the repo — I fixed it by using `git mv` to flatten the structure while preserving Git history, rather than just deleting and re-adding files."*

Problem 2: Direct push to `main` got rejected

What happened: After fixing the folder structure, `git push origin main` failed with:

```
remote: error: GH006: Protected branch update failed for refs/heads/main.  
remote: - Changes must be made through a pull request.
```

Root cause: Branch protection was working as intended — direct pushes to `main` are blocked once "Require a pull request before merging" is enabled.

Fix: Created a new branch (`fix/flatten-repo-structure`), pushed that branch instead, then opened a Pull Request into `main`.

Interview line: *"This wasn't a bug — it was the branch protection rule doing its job. It reinforced why PR-based workflows exist: they prevent someone (including yourself) from accidentally pushing directly to the stable branch."*

Problem 3: PR merge blocked by "Require approvals"

What happened: After opening the PR, GitHub showed **"Merging is blocked — at least 1 approving review is required."** Since this is a solo project, there was no second person to approve it, and GitHub doesn't allow self-approval by default.

Root cause: The branch protection rule had "Require approvals" enabled, which is meant for team environments with multiple reviewers.

Fix: Edited the branch protection rule to keep "Require a pull request before merging" ON (so the PR/CI workflow is still enforced), but turned OFF "Require approvals" (since it doesn't apply to solo work). This still requires PRs and CI checks, just not a second human reviewer.

Interview line: *"I ran into a case where the required-approval rule made sense for teams but not for solo work — I adjusted the protection rule to keep automated checks and the PR requirement, while removing the review requirement that doesn't apply when working alone."*

4. Standard Workflow Going Forward

For every new feature/task from here on:

```
git checkout dev
git pull origin dev
git checkout -b feature/<short-task-name>

# ... write code, commit in small logical chunks ...
git add .
git commit -m "feat: <description>"
git push origin feature/<short-task-name>

# On GitHub: open PR into dev → wait for CI checks → merge
# Periodically: open PR from dev → main once stable
```

Commit message convention used: **Conventional Commits** (feat:, fix:, docs:, chore:, etc.) — makes history easy to read and is a widely used industry standard.

5. Resume/Interview Summary (one-liner)

"Set up a production-style project repository with a microservices-oriented folder structure, protected branches, PR-based Git workflow, and automated CI checks via GitHub Actions — solving real issues like folder structure mismatches and branch protection conflicts along the way."